

Received 8 July 2024, accepted 18 July 2024, date of publication 24 July 2024, date of current version 1 August 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3432979

RESEARCH ARTICLE

Multiplication-Free Lookup-Based CNN Accelerator Using Residual Vector Quantization and Its FPGA Implementation

HIROSHI FUKETA[✉], (Member, IEEE), TOSHIHIRO KATASHITA, YOHEI HORI, (Member, IEEE), AND MASAKAZU HIOKI, (Member, IEEE)

National Institute of Advanced Industrial Science and Technology (AIST), Tsukuba, Ibaraki 305-8568, Japan

Corresponding author: Hiroshi Fuketa (h-fuketa@aist.go.jp)

This work was supported in part by JSPS KAKENHI under Grant JP24K02918.

ABSTRACT In this paper, a table lookup-based computing technique is proposed to perform convolutional neural network (CNN) inference without multiplication, and its FPGA implementation is demonstrated as a proof-of-concept. Conventionally, the hardware specific to the lookup-based dot product approximation (LDA) has been proposed to achieve energy-efficient CNN computations for edge AI applications. However, it has not been applied to complicated AI tasks, such as ImageNet image classification, because LDA degrades the inference accuracy, especially in complicated tasks. Therefore, a new LDA technique using residual vector quantization, called RLDA, is proposed in this study. By adopting the proposed RLDA to CNN, we can achieve an inference accuracy degradation of below 5% for ResNet-18 model on ImageNet 1000 classification. In addition, the proposed RLDA-based CNN accelerator is implemented on a ZCU104 evaluation board, which includes a Zynq UltraScale+ FPGA and DDR4 DRAM. We compare the processing time of the proposed accelerator on ZCU104 with that of NVIDIA Jetson AGX Orin and reveal that the processing time of the proposed accelerator is comparable to that of Jetson Orin in the lower layers of the ResNet-18 model. Finally, the computational performance of the proposed accelerator is compared with conventional FPGA-based accelerators. The proposed accelerator does not require a digital signal processor (DSP), which is often available in modern FPGAs. We demonstrate that the proposed accelerator achieves a performance more than four times higher than that of the conventional FPGA-based accelerator without DSP.

INDEX TERMS Associative memory, computing-in-memory, convolutional neural network, FPGA, vector quantization.

I. INTRODUCTION

Traditionally, AI computation has been conducted in the cloud, whereas AI processing near users, called “edge AI,” is becoming an emerging research topic these days [1]. For edge AI, energy efficiency (performance per power) is a key metric because edge devices are often battery operated. For example, NVIDIA A100, which is an AI accelerator for cloud

computing, achieves 19.5 TFLOPS per 300 W [2], whereas NVIDIA Jetson AGX Orin, which is an AI accelerator for edge AI, reaches 5.3 TFLOPS at 15-60 W [3]. Because AI algorithms will become increasingly complicated, that is, the amount of AI computation will become increasingly significant, further improvements in energy efficiency are required.

Computing-in-memory (CIM) is considered as one of promising solutions to achieve higher energy efficiency [4], [5], [6], [7]. Most AI systems and services today are based on neural networks (NNs), and NN computation mainly consists

The associate editor coordinating the review of this manuscript and approving it for publication was Ilaria De Munari[✉].

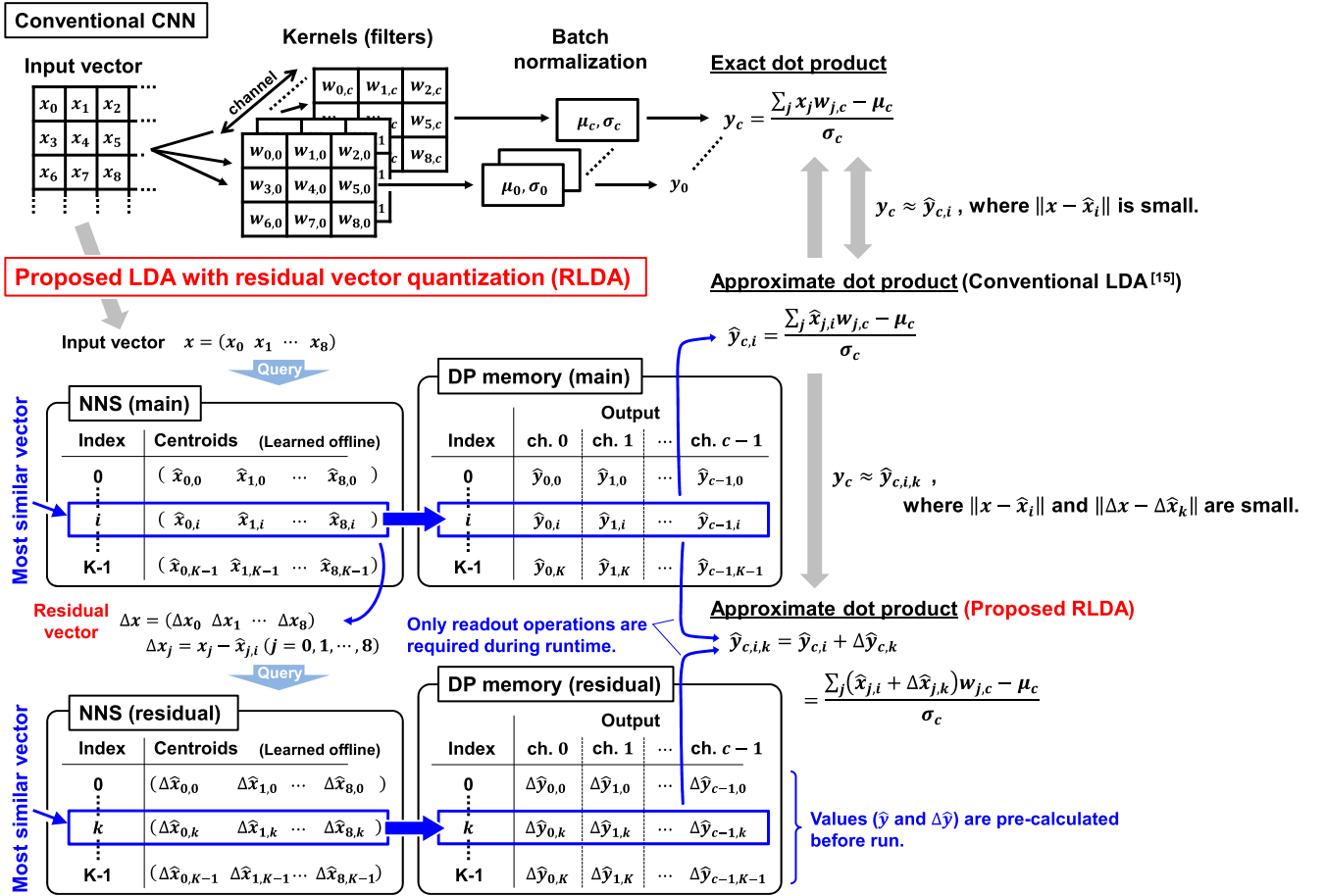


FIGURE 1. The overview of the proposed lookup-based dot product approximation using residual vector quantization (RLDA). When RLDA is adopted to CNN (RLDA-based CNN), computations of CNN are performed with a nearest neighbor search (NNS) and read operation of memory storing pre-calculated dot products (DP memory). When the Manhattan distance is used as a distance metric of NNS, multiply operations are not required in CNN computations.

of a large number of multiply-accumulate (MAC) operations. In the conventional AI accelerators mentioned above, values (typically input activations and weights) are moved from the memory to MAC units, and then the results of MAC operations are written back to memory. On the other hand, in the CIM, MAC operations are executed in the memory; hence, the energy of data movement between memory and MAC units can be reduced.

To improve the energy efficiency beyond CIM, we focus on MAC operations. In MAC operations, multiplication is more energy-hungry than addition [8]. Therefore, eliminating the multiplication operations in MAC is the best way to further improve the energy efficiency of AI accelerators. Based on this approach, table lookup-based computing techniques for NN using vector quantization (VQ) [9] and product quantization (PQ) [10] have been proposed in [11], [12], [13], [14], and [15]. PECAN [11] and LUT-NN [12] are techniques to eliminate multiplication by substituting MAC operations in NN with a nearest neighbor search (NNS) and memory read (table lookup) operations. They are validated on CIFAR100 and ImageNet image classification tasks, respectively. Although NNS and table lookups can be accelerated by a specific hardware, they have not been discussed in

these literatures. As such specific hardware, FPGA and ASIC implementations are demonstrated in [13], [14], and [15]. However, they process simpler tasks such as MNIST and CIFAR10 image classification, and the feasibility of adopting them for more complicated tasks such as ImageNet image classification remains unclear.

In this paper, we propose a multiplication-free NN accelerator and implement it on FPGA. To the best of our knowledge, this is the first study to demonstrate the specific hardware implementing table lookup-based computing that can process ImageNet 1000 image classification without multiplication. The proposed NN accelerator is based on the lookup-based dot product approximation (LDA) proposed in [15]. LDA is more efficient when it is adopted in convolutional neural networks (CNNs); hence, in this study, we focus on the case in which LDA is applied to CNN (LDA-based CNN).

The challenge of the conventional LDA-based CNN accelerator is that LDA hardly degrades the inference accuracy for simpler tasks such as MNIST and CIFAR10 image classification, whereas it significantly affects the accuracy for complicated tasks such as ImageNet image classification [15]. This is because 1) the quantization error of VQ in LDA is not ignorable, and 2) the analog computation-based

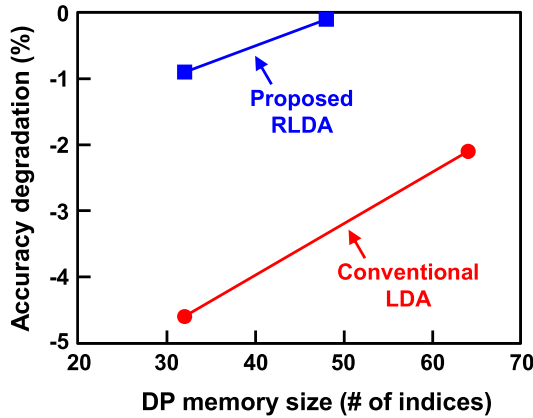


FIGURE 2. Comparison of accuracy degradation due to the conventional LDA and proposed RLDA when ResNet-9 model is used for image classification on CIFAR-10 dataset. As a metric of memory size, the number of indices of the DP memory (K in Fig. 1) is used.

associative memory is used for the NNS in the conventional LDA-based accelerator, and its search accuracy is not 100 %, which affects the inference accuracy. To solve these two problems, in this study, we propose that 1) a new LDA technique using a residual vector quantization [16], [17], called RLDA, is adopted to reduce the quantization error, as depicted in Fig. 1, and 2) NNS is realized using a fully digital circuit, which achieves a search accuracy of 100 %. Consequently, we can achieve an inference accuracy degradation of less than 5 % for the ResNet-18 model on ImageNet 1000 classification. Subsequently, we implement the RLDA-based CNN accelerator on FPGA and demonstrate that its processing time is comparable to that of NVIDIA Jetson Orin, which is a conventional edge AI accelerator, in the lower layers of the ResNet-18 model. In addition, the proposed accelerator is compared with the conventional FPGA-based accelerators. Because the proposed accelerator can perform CNN computations without multiplication, a digital signal processor (DSP), which is often available in modern FPGAs, is not required. This is preferable for mitigating from FPGA to ASIC. In this study, we demonstrate that the computational performance of the RLDA-based CNN accelerator implemented on FPGA is four times higher than that of the conventional FPGA-based accelerator without DSP.

The remainder of this paper is organized as follows. The details of the proposed RLDA-based CNN accelerator are presented in Section II. Its FPGA implementation is described in Section III. Then, the measurement results are presented and discussed in Sections IV and V, respectively. Finally, Section VI concludes this paper.

II. DOT PRODUCT APPROXIMATION USING RESIDUAL VECTOR QUANTIZATION

A. ALGORITHM

The LDA was proposed in [15] to achieve efficient CNN processing. The concept of LDA is illustrated in Fig. 1. CNN processing primarily comprises numerous MAC operations.

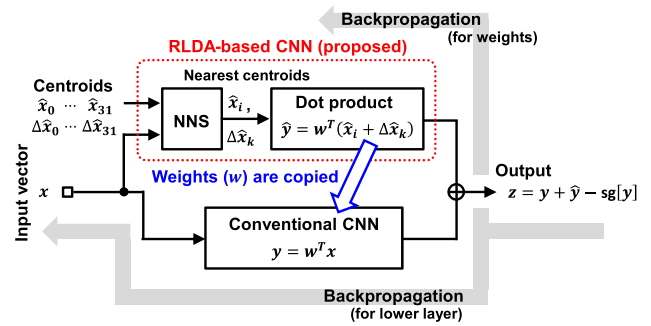


FIGURE 3. Proposed training method for the RLDA-based CNN. This is based on the method proposed in [15], whereas the backpropagation path is slightly modified. $sg[\cdot]$ denotes the stop gradient operator presented in [20].

The energy required to execute multiplication is much larger than that required to execute addition [8]. LDA substitutes MAC operations with addition and memory read operations; that is, it removes multiplication operations, thus reducing the energy of CNN processing [15]. However, we found that the inference accuracy of the LDA-based CNN deteriorates particularly in complicated tasks such as the ImageNet benchmark (image classification). Therefore, in this study, we propose a new LDA technique using a residual vector quantization (RLDA).

In LDA, the dot product of vectors x and w can be approximated as follows [9],

$$w^T x \approx w^T \hat{x}, \quad (1)$$

where $\|x - \hat{x}\|$ is sufficiently small. In the residual vector quantization [16], [17], a residual vector Δx is defined as,

$$\Delta x = x - \hat{x}. \quad (2)$$

In the proposed RLDA, the dot products of vectors x and w can be approximated as follows,

$$w^T x \approx w^T (\hat{x} + \Delta \hat{x}), \quad (3)$$

where $\|x - \hat{x}\|$ and $\|\Delta x - \Delta \hat{x}\|$ are sufficiently small.

In the proposed RLDA-based CNN, $\hat{x}$ and $\Delta \hat{x}$ are called centroids, which are obtained in the training phase. In addition, dot products of the centroids and weights (w) are also calculated during the training phase ($\hat{t} = w^T \hat{x}$ and $\Delta \hat{t} = w^T \Delta \hat{x}$ in Fig. 1). In the inference, the indices of the centroids, which are most similar to the input and residual vectors, are determined by a nearest neighbor search (NNS). Then, the pre-calculated dot products stored in the DP memories are obtained by a table lookup using the indices, as depicted in Fig. 1. In this study, the Manhattan distance is used as a distance metric of the NNS, which can be calculated by subtraction and addition only. Thus, in the proposed RLDA, dot products can be approximately calculated by add/sub-operations and memory readouts, and multiplications are not required.

Fig. 2 depicts the comparison between the conventional LDA and proposed RLDA. In this figure, the conventional LDA and proposed RLDA are applied to compute the

ResNet-9 model for image classification on the CIFAR-10 dataset [15]. Although RLDA requires twice as much NNS and memory readout as LDA, the number of centroids (K in Fig. 1) required for RLDA is smaller than that required for LDA to achieve the same inference accuracy. This is because the residual vector quantization used in the proposed RLDA can reduce quantization errors compared to VQ used in the conventional LDA. Consequently, the size of the DP memory, which is the memory storing the pre-calculated dot products (Fig. 1), can be reduced compared to LDA, as shown in Fig. 2.

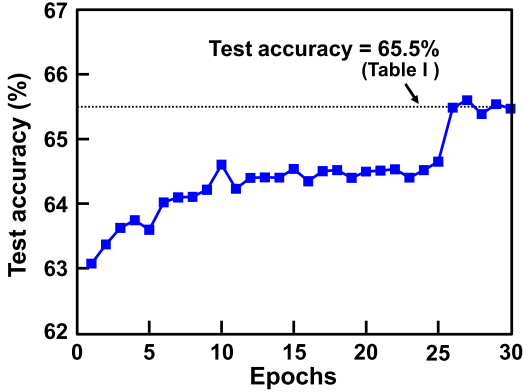


FIGURE 4. Test accuracy of ImageNet 1000 classification (top-1) in the additional training when the proposed RLDA is applied to the ResNet-18 model.

TABLE 1. Top-1 Test accuracy of ImageNet 1000 classification.

	precision	Inference accuracy
Conventional	FP32	69.4 %
Proposed RLDA	FP32	65.5 %
	INT8 (This work)	64.5 %

B. TRAINING METHOD

In this study, the target network is ResNet-18 for ImageNet [18]. ResNet-18 comprises 17 CNN layers and one fully connected layer. The proposed RLDA accelerator processes 16 CNN layers, excluding the first and last layers.

Additional training is required for the RLDA-based CNN after training the conventional ResNet-18 model. The proposed RLDA consists of NNS and memory readout, as shown in Fig. 1. These operations are not differentiable. Therefore, the RLDA-based CNN cannot be trained in a conventional manner. In the previous work [15], a training method for the LDA-based CNN using straight-through estimator (STE) [19] has been proposed. In this study, we modify it slightly such that the weights are updated by backpropagation using approximate dot products, as illustrated in Fig. 3. After backpropagation, the updated weights are copied to the weights of the conventional CNN (Fig. 3), that is, the weights of the conventional CNN are not updated by backpropagation.

The training procedure for the proposed RLDA-based CNN conducted in this study is as follows:

TABLE 2. The number of channels and data size.

(a) Original ResNet-18					
	# of input channel	# of output channel	Parameter size (kB) (Conv.)	Data size (kB)	
				Centroids	DP memory
layer 2	64	64	144	36	256
layer 3	64	64	144	36	256
layer 4	64	64	144	36	256
layer 5	64	64	144	36	256
layer 6	64	128	288	36	512
layer 7	128	128	576	72	1,024
layer 8	128	128	576	72	1,024
layer 9	128	128	576	72	1,024
layer 10	128	256	1,152	72	2,048
layer 11	256	256	2,304	144	4,096
layer 12	256	256	2,304	144	4,096
layer 13	256	256	2,304	144	4,096
layer 14	256	512	4,608	144	8,192
layer 15	512	512	9,216	288	16,384
layer 16	512	512	9,216	288	16,384
layer 17	512	512	9,216	288	16,384
Total			42,912	1,908	76,288

(b) ResNet-18 compressed by CHEX [21] (this work)					
	# of input channel	# of output channel	Data size (kB) (This work)		
			Centroids	DP memory	
layer 2	64	43	36	172	
layer 3	43	64	24	172	
layer 4	64	44	36	176	
layer 5	44	64	25	176	
layer 6	64	98	36	392	
layer 7	98	90	55	551	
layer 8	90	89	51	501	
layer 9	89	109	50	606	
layer 10	109	193	61	1,315	
layer 11	193	174	109	2,099	
layer 12	174	163	98	1,773	
layer 13	163	214	92	2,180	
layer 14	214	394	120	5,270	
layer 15	394	409	222	10,072	
layer 16	409	364	230	9,305	
layer 17	364	481	205	10,943	
Total			1,450	45,703	

1. The original ResNet-18 model (i.e., RLDA is not applied) is trained in a conventional manner.
2. Using the model trained in Step 1, inference is performed in one epoch of the training dataset. During this inference, the centroids $\hat{x}$ and $\Delta\hat{x}$ in each layer of ResNet-18 model are obtained using the k-means clustering. In this study, the number of centroids (K in Fig. 1) is set to 32.
3. Additional training for the RLDA-based CNN using the training method depicted in Fig. 3 is performed. The weights obtained in Step 1 and centroids obtained in Step 2 are used as the initial values. The centroids are updated using the exponential moving averages (EMA) [20] during the additional training. The number of epochs of this additional training is 30. The learning rate is 10^{-3} in the first 25 epochs and 10^{-4} in the remaining 5 epochs. The batch size is set to 30.

Please note that the compressed ResNet-18 model (details are provided in Section II-C) is used in this study. The test accuracy of the ImageNet 1000 classification (top-1) in the additional training is shown in Fig. 4. The final test accuracy values are listed in Table 1. The accuracy of the conventional CNN with the compressed ResNet-18 model obtained in Step 1 of the abovementioned training procedure is 69.4 % with 32-bit floating point (FP32) precision. When the proposed RLDA is adopted, the accuracy deteriorates by -3.9 %. The model is trained using Python (ver. 3.10) and PyTorch (ver. 2.0).

C. QUANTIZATION AND MODEL COMPRESSION

The proposed RLDA (Fig. 1) does not depend on number representation. Considering hardware implementation, it is considerably more area- and energy-efficient when integer number representations are used. Therefore, in this study, the input data (feature map), centroids, and pre-calculated dot products stored in the DP memories (Fig. 1) are quantized to 8-bit integer (INT8). The quantization procedure used in this study is as follows:

1. FP32 precision is used in the training phase described in Section II-B.
2. The centroids and pre-calculated dot products obtained in Step 1 is quantized from FP32 to INT8.
3. Only the weights in the last layer of the ResNet-18 model (fully connected layer) are fine-tuned.

The test accuracy of the ImageNet 1000 classification when the quantization procedure described above is conducted is listed in Table 1. The inference degradation caused by the quantization is -1.0 %.

In the conventional CNN processing, the data movement of weight parameters (w in Fig. 1) from DRAM is required. On the other hand, the data movement of centroids ($\hat{x}$ and $\Delta\hat{x}$ in Fig. 1) and pre-calculated dot products ($\hat{t}$ and $\Delta\hat{t}$ in Fig. 1) instead of the weights is required in the proposed RLDA. Thus, it is important to reduce the data size of the centroids and pre-calculated dot products, and quantization is also effective to achieve it. Because the data size depends on the number of channels in CNN, a CNN model compression technique, called CHEX [21], which can reduce the number of channels in CNN without inference accuracy degradation, is beneficial for the RLDA. Thus, we adopt this technique in this study. The number of input and output channels of each layer in the original ResNet-18 and the compressed model are listed in Table 2(a) and (b), respectively. The data size of the weight parameters (FP32 precision) is also listed in Table 2(a). In addition, the data sizes of the centroids and DP memories (Fig. 1) when the centroids and pre-calculated dot products stored in the DP memories are quantized to INT8 are listed. The total data size of the weight parameters in the original ResNet-18 model is 42 MB (Table 2(a)), whereas the total size of the centroids and DP memories the compressed ResNet-18 model is 46 MB (Table 2(b)). This means that the data size required in the proposed RLDA is 10% larger than that required in the conventional CNN processing.

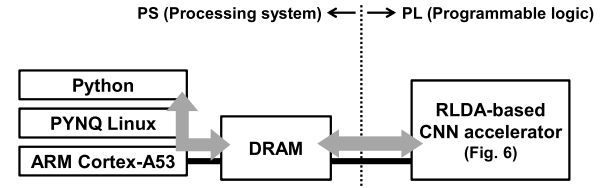


FIGURE 5. Implementation overview of the proposed RLDA-based CNN accelerator. The FPGA chip used in this work consists of PS and PL. The accelerator is implemented on PL. The software for image recognition written by Python runs on PS (PYNQ platform) and CNN processing in it is offloaded to the accelerator on PL via DRAM.

III. FPGA IMPLEMENTATION OF RLDA-BASED CNN ACCELERATOR

In this study, we use a ZCU104 evaluation board [22] to implement the proposed RLDA-based CNN accelerator. This evaluation board includes a Zynq UltraScale+ XCZU7EV MPSoC FPGA chip and 2 GB DDR4 DRAM. The FPGA chip consists of an ARM Cortex-A53, called processing system (PS), and programmable logic (PL). The proposed RLDA-based CNN accelerator is implemented on the PL. The PYNQ platform is used for ease of development (as depicted in Fig. 5). Most of the CNN processing is offloaded to the RLDA-based CNN accelerator on the PL via the DRAM. The offloading flow is controlled by the Python program on the PYNQ platform.

Fig. 6 shows the overall architecture of the proposed RLDA-based CNN accelerator. The data (feature map) and centroids are moved to the working memory and codebook memory, respectively, from DRAM. As an activation function, ReLU unit is implemented. The processing elements (PEs) are the main cores that execute the proposed RLDA explained in Fig. 1. Each PE reads the data (feature map) and centroids from the working memory and codebook memory, respectively, for each input channel, and PE outputs the pre-calculated dot products required in CNN processing. The dot products are summed output channel by channel in the adder tree. The (partial) sums are written back to the working memory.

The detailed architecture of the PE is illustrated in Fig. 7. In the PE, the input data (feature map) of one input channel in each layer of the ResNet-18 model is read from the working memory (Fig. 6) and temporarily stored in the data buffer. Then, the pre-calculated dot products between the centroids and weights are moved from the DRAM to the DP memories (Fig. 1). In this study, the DP memory is realized using 19 block RAMs (BRAMs). The unit size of one BRAM is 72 b/word; hence, the DP memory has large bits per word of 1344. This means that the DP memory can store the dot products of 168 output channels (i.e., $c = 168$ in Fig. 1) with INT8 precision. During CNN processing, several parts of the data (feature map) are fetched from the data buffer to the line buffer. The line buffer comprises shift registers that realize the kernel movement in CNN. The data in the kernel is input into the circuits for conducting the NNS (NNS core in Fig. 7). The PE contains two NNS cores to realize the residual vector quantization. The NNS core outputs the index (i and k in

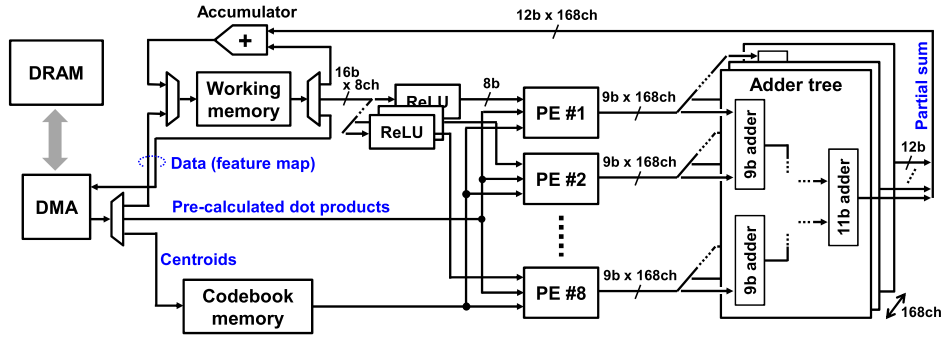


FIGURE 6. Overall architecture of the proposed RLDA-based CNN accelerator. The accelerator contains eight PEs explained in Fig. 7.

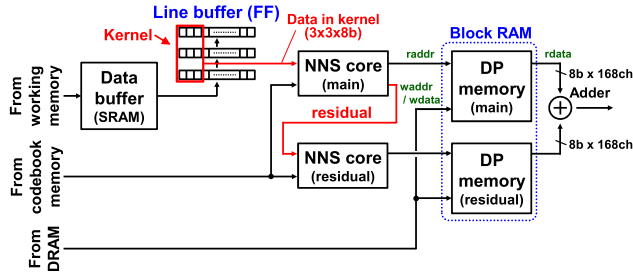


FIGURE 7. Architecture of PE (Fig. 6). PE consists of two NNS cores (Fig. 8) and DP memories (Fig. 1) with buffers for the input data (feature map).

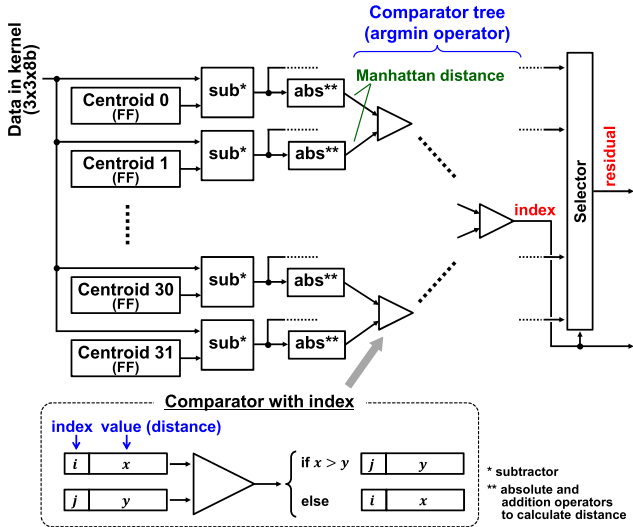
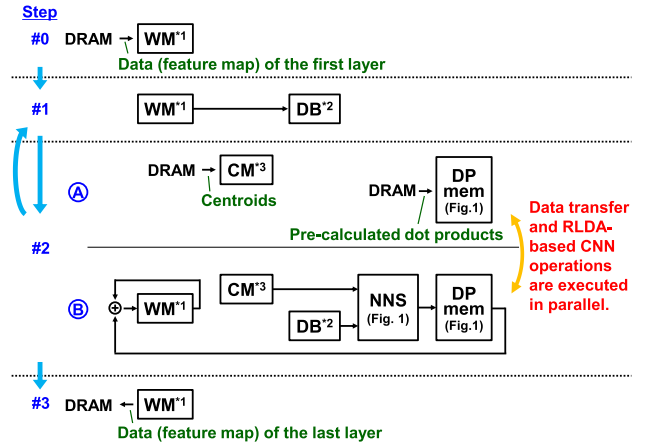


FIGURE 8. Architecture of NNS core (Fig. 7). NNS core returns the index of centroids which are the most similar to the input vector (kernel data) and residual vector. In this work, the Manhattan distance is used as the metrics of similarity. The number of centroids (K in Fig. 1) is 32 in this work, as explained in Section II-B.

Fig. 1) of the vector that is most similar to its input. By using the index as a read address of the DP memory, we can obtain the approximate dot products between the data in the kernel and weights.

The NNS core contains memory for the centroids, subtractors, absolute operators, and comparator tree, as depicted in Fig. 8. The Manhattan distance between the input data and each centroid is calculated using the subtractors and absolute operators. As illustrated in Fig. 8, the inputs of the comparator



*1 WM: Working memory (Fig. 6) *2 DB: Data buffer in PE (Fig. 7) *3 CM: Codebook memory (Fig. 6)

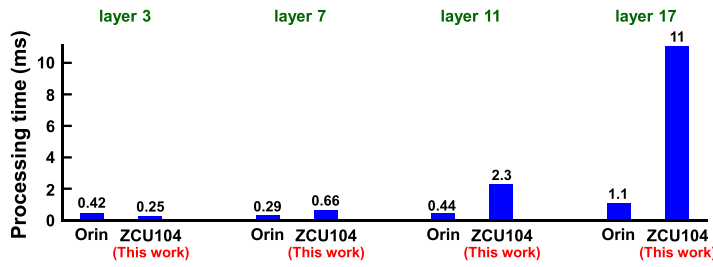
FIGURE 9. Processing sequence of the proposed RLDA-based CNN.

TABLE 3. Utilization of PL when the proposed RLDA-based CNN accelerator is implemented on ZCU104 (XCZU7EV FPGA).

Resource	Available	Used	Utilization (%)
LUT	230,400	203,164	88.2
FF	460,800	153,677	33.4
BRAM	312	307	98.4
DSP slice	1,728	0	0

in the comparator tree are two values (distances) along with their indices. The comparator then outputs a smaller value (distance) along with its index. Thus, the comparator tree can realize argument minimum (argmin) function.

Fig. 9 shows the processing sequence of the proposed RLDA-based CNN accelerator. First, the data of the first layer in the ResNet-18 model is transferred from the DRAM to the working memory in the proposed accelerator (Fig. 6). Subsequently, in Step #1 of Fig. 9, the data is moved to the data buffers in PEs (Fig. 7). After that, the centroids and pre-calculated dot products between the centroids and weights are transferred from the DRAM to the codebook memory (Fig. 6) and DP memories in the PEs (Fig. 7), respectively (Step #2-A in Fig. 9). Simultaneously, the RLDA-based CNN operations are executed (Step #2-B in Fig. 9). To achieve such simultaneous operation, dual-port



	# of input channel	# of output channel	# of MACs (MOPS)	Peak Performance (TOPS)	Codebook & DP memory size (kB)
layer 3	43	64	155	1.4	194
layer 7	98	90	124	1.9	606
layer 11	193	174	118	3.6	2,208
layer 17	364	481	154	3.6	11,148

FIGURE 10. Processing time required by Nvidia Jetson Orin and the proposed RLDA-based CNN accelerator implemented on ZCU104 evaluation board to perform CNN computations in each layer of the ResNet-18 model. The batch size is one.

TABLE 4. Performance summary.

	Jetson Orin [3]	RLDA-based CNN accelerator on ZCU104 (This work)
Implementation	8nm ASIC	16nm FPGA
Clock speed (MHz)	1300	150
DRAM bandwidth (GB/s)	204.8	20 (ideal ^{(*)1}) 1 (measured)
Peak performance	5.3 TFLOPS ^{(*)2}	1.4 TOPS ^{(*)3} (layer 3) 3.6 TOPS ^{(*)3} (layer 10~17)
Power (W)	15 - 60	4.2 (PL) / 2.8 (PS)

(^{(*)1}) Theoretical bandwidth of DDR4-2400 used in ZCU104.
(^{(*)2}) FP32 precision (^{(*)3}) INT8 precision

TABLE 5. Theoretical performance per PE of proposed accelerator.

(a)	Peak performance (OPS)	$f \times S \times c \times 2^{*1}$
(b)	Amount of computation (OP)	$H \times W \times S \times c \times 2^{*1}$
(c)	Calculation time (s) [= (b)/(a)]	$\frac{H \times W}{f}$
(d)	Data preparation time (s)	$\frac{H \times W}{f}$
(e)	Transferred data from DRAM ^{(*)2} (Byte)	$K \times c \times 2$
(f)	Required DRAM Bandwidth [= (e) / ((c)+(d))] (Byte/s)	$\frac{f \times K \times c}{H \times W}$

f : Clock frequency (Hz) S : Kernel size (= 9 in ResNet-18)
 c : The number of output channels (Fig. 1)
 K : The number of centroids (Fig. 1)
 H, W : Height and width of input feature map
 (= 56 in layers 2-6, 28 in layers 7-10, 14 in layers 11-14,
 7 in layers 15-17 in ResNet-18 for ImageNet)

(^{(*)1}) MAC operation consists of 2 operations (=ADD+MULT).

(^{(*)2}) Centroids and pre-calculated dot products must be transferred.

The number of pre-calculated dot products are much larger than that of centroids, as shown in Table II(b). Thus, pre-calculated dot products are only considered here.

memories are used for the codebook memory and DP memories in PEs. When all operations are completed, the step returns to #1, and later the next layer of the ResNet-18 model is processed.

The utilization of the PL after implementing the proposed RLDA-based CNN accelerator on ZCU104 is listed in Table 3. The FPGA implementation is developed using Vivado (ver. 2019.1). As shown in Figs. 6-8, the proposed RLDA-based CNN accelerator can perform CNN computations with addition and memory readout, which are realized using LUTs, FFs, and memories without DSP. The single PE requires around 20k LUTs. Thus, we decided to set the number of PEs to eight, as shown in Fig. 6, by considering the total number of LUTs available in the XCZU7EV FPGA chip (230k LUTs). This means that the RLDA-based CNN accelerator can simultaneously process the eight input channels. In the proposed RLDA, it is better to maximize the utilization of BRAMs, which are primarily used for the DP memories that store the pre-calculated dot products to achieve higher computational performance. Thus, we decided to use 19 BRAMs for the DP memory, as mentioned previously, that is, 38 BRAMs per PE (PE includes two DP memories, as shown in Fig. 7). Consequently, the BRAM utilization reaches 98.4 %, as listed in Table 3.

IV. MEASUREMENT RESULTS

The proposed RLDA-based CNN accelerator implemented on the ZCU104 evaluation board is compared with the NVIDIA Jetson AGX Orin [3], as we focus on edge AI in this study. A performance summary of Jetson Orin and

the proposed accelerator is listed in Table 4. The proposed accelerator implemented on the XCZU7EV FPGA chip can operate at 150 MHz. The peak performance per PE is expressed as Eq. (a) in Table 5. As indicated in this equation, the peak performance depends on the number of output channels (c in Fig. 1). In the proposed RLDA, the number of output channels is determined by the bits per word in the DP memory, as shown in Fig. 1. In this work, the bits per word in the DP memory is 1344, which corresponds to 168 output channels, as described in Section III. Thus, the performance per PE reaches 0.45 TOPS/PE according to Eq (a) in Table 5. In this work, eight PEs are implemented as shown in Fig. 6. Consequently, a theoretical (maximum) peak performance of 3.6 TOPS is achieved. This means that the maximum performance depends on the size of the DP memories (Fig. 1) realized by BRAMs. As listed in Table 3, 307 BRAMs (11 Mb) corresponding to almost all BRAMs implanted in this FPGA are used for the DP memories. In the layer with the smaller number of output channels than 168, the performance becomes worse than the peak performance of 3.6 TOPS according to Eq (a) in Table 5. For example, the number of

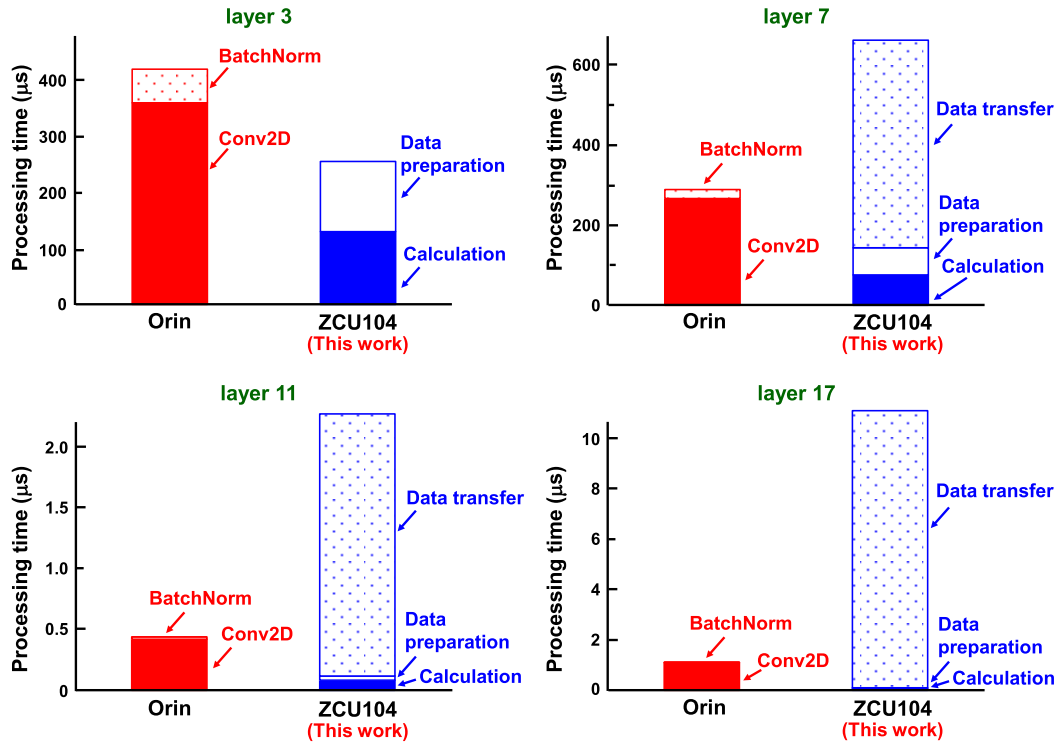


FIGURE 11. Breakdown of processing time shown in Fig. 10.

output channels in layer 3 is 64. Therefore, a theoretical peak performance in layer 3 is 1.4 TOPS, as listed in Table 4. According to the report generated by the Vivado tool, the power dissipation of the PL is estimated to 4.2 W.

The time required by the proposed RLDA-based accelerator to perform CNN computations in each layer of the compressed ResNet-18 model (Table 2(b)) is compared in Fig. 10. The batch size is set to one, assuming the use case of image classification at the edge. In Jetson Orin, the ResNet-18 model is processed in a conventional manner (i.e., RLDA is not adopted). The processing time is measured using a profiler (NVIDIA Nsight Systems [23]). In this study, we define the processing time of Jetson Orin as the sum of durations, in which the convolution (denoted by “conv2d” in the profiling result) and batch normalization (denoted by “batch_norm” in the profiling result) functions operate, shown in “CUDA HW” in the profiling result. On the other hand, to measure the processing time of the proposed RLDA-based CNN accelerator, the number of clocks taken for Steps #1 and #2 in Fig. 9 is counted. Please note that batch normalization function is not required for the proposed accelerator as it is included in the pre-calculated dot products, as shown in Fig. 1. This is an advantage of the proposed RLDA.

Fig. 10 indicates that the processing time of the proposed RLDA-based CNN accelerator on ZCU104 in the lower layers are comparable to that of Jetson Orin, whereas that of ZCU104 is longer than that of Jetson Orin in the upper layers.

To discuss the reason, we present a breakdown of the processing time in Fig. 11. As mentioned above, the processing time of Jetson Orin consists of time required for the convolution operation denoted by “Conv2d” in Fig. 11 and the batch normalization function denoted by “BatchNorm” in Fig. 11. In contrast, the processing time of the proposed accelerator on the ZCU104 divided into the following three parts:

- The **data preparation** time is the time required to move the data from the working memory (Fig. 6) to data buffer in the PE (Fig. 7), which corresponds to Step #1 in the processing sequence of the RLDA shown in Fig. 9. This time is expressed as Eq. (d) in Table 5.
- The **data transfer** time is the time required to transfer the centroids and pre-calculated dot products of the centroids and weights from the DRAM, which corresponds to Step #2-A in the sequence shown in Fig. 9. The amount of the transferred data is expressed as Eq. (e) in Table 5.
- The **calculation** time is the time required to execute the RLDA-based CNN operations (Fig. 1), which corresponds to Step #2-B in the sequence shown in Fig. 9. This time is expressed as Eq. (c) in Table 5.

The centroids and pre-calculated dot products are transferred from the DRAM in parallel with the RLDA-based CNN operations, as shown in Fig. 9 (Steps #2-A and B). However, Table 2 indicates that the layer is higher, the data size of centroids and pre-calculated dot products becomes larger. Thus, there is no data transfer time in layer 3 (the data is

TABLE 6. Comparison of FPGA-based NN accelerators.

	TCAS23 [24]	ISCAS17 [25]	ACCESS21 [26]	This work
FPGA	AMD Ultrascale+ XCZU9EG	Altera Arria 10 GX 1150	AMD Virtex 7 VC709	AMD Ultrascale+ XCZU7EV
Process (nm)	16	20	28	16
Target model	ViT-Small	ResNet-50	GoogLeNet	ResNet-18
Precision	INT8	INT16	INT12	INT8
Clock frequency (MHz)	300	150	200	150
DSP	Used	Used	Not used	Not used
Measured Performance (GOP/s)	1037^{*1}	285	129	620^{*2}

(*1) Performance to compute the QKV operation (fully connected layer).

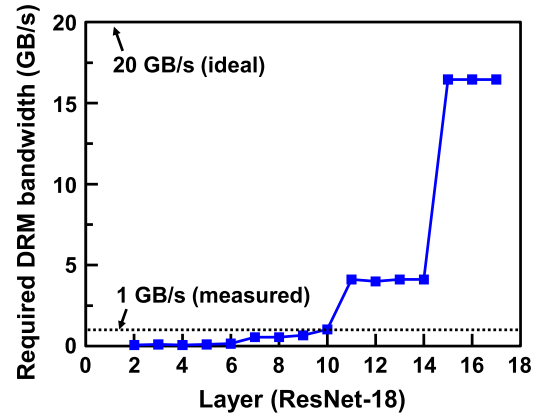
(*2) Performance of layer 3 including data transfer and preparation time (Fig. 11).

transferred in the background of the RLDA-based CNN operations perfectly), whereas most of the processing time is spent on the data transfer from DRAM in layer 17 because the data transfer must be completed before starting the RLDA-based CNN operations, which causes the waiting time. This reason will be discussed later in Section V.

Finally, Table 6 lists a comparison of the proposed RLDA-based CNN accelerator with conventional FPGA-based NN accelerators for image recognition [24], [25], [26]. DSP is often available in modern FPGAs. By utilizing DSP, a higher performance than that of the proposed accelerator has been demonstrated in [24]. However, DSP is not preferable considering the migration from FPGA to ASIC. The RLDA-based accelerator proposed in this study does not use DSP, as indicated in Table 3. It achieves a performance of more than four times higher than that of the conventional DSP-free accelerator [26]. The measured performance in layer 3 is 620 GOPS, as shown in Table 6, and it is smaller than the theoretical performance (1.4 TOPS) in Table 4. This is because the data preparation time is required in addition to the calculation time, as shown in Fig. 11 (layer 3), and the measured performance includes the data preparation time, whereas the theoretical performance does not include it.

V. DISCUSSION AND FUTURE WORK

As shown in Fig. 11, the processing time of the proposed RLDA-based CNN on ZCU104 is mainly occupied by the data transfer time from the DRAM, especially in the higher layers of the ResNet-18 model. This occurs when the data transfer time (Eq. (e) in Table 5) is larger than the calculation time (Eq. (c) in Table 5), as described in Section IV. To investigate this quantitatively, the DRAM bandwidth required to achieve the data transfer time smaller than the calculation time is derived as Eq. (f) in Table 5. Fig. 12 shows the dependence of the required DRAM bandwidth on the layer in ResNet-18. The effective DRAM bandwidth is estimated to 1 GB/s from the measurement results. In this case, the required DRAM bandwidth is not satisfied in the higher layer, as shown in Fig. 12. This is consistent with Fig. 11. The measured DRAM bandwidth of 1 GB/s is considerably slower than the ideal bandwidth of the DDR4 DRAM on the ZCU104

**FIGURE 12.** DRAM bandwidth required to achieve the data transfer time smaller than the calculation time (Eq. (f) in Table 5).

(20 GB/s), as listed in Table 4. We understand that this is because DRAM is shared with PS and PL; that is, the PS-side DRAM is used. In the ZCU104, a DRAM that is directly connected to the PL can be added. If this PL-side DRAM is used, the DRAM bandwidth will increase; consequently, the processing time of the proposed RLDA-based CNN on ZCU104 will be comparable to that of Jetson Orin even in the higher layers of CNN.

The inference accuracy of the proposed RLDA-based CNN deteriorates compared to that of the baseline (conventional CNN without RLDA), as listed in Table 1. It is another future work to mitigate the accuracy degradation. This will be achieved by improving the training algorithm. For example, Tang et al. recently proposed a lookup-based CNN using NNS realized by softmax function, which enables backpropagation, and demonstrated that better inference accuracy can be achieved [12].

In this paper, the kernel size is fixed to 3×3 . Thus, the proposed implementation does not support the models that have layers with other kernel sizes, such as ResNet-50. However, it is possible to support other kernel size by changing the number of elements of centroid vectors in Fig. 1.

In this study, the proposed RLDA-based CNN accelerator is implemented on FPGA as a proof-of-concept. If it is implemented on ASIC, the computational performance and energy efficiency will be improved. Especially, the peak performance is proportional to the number of output channels, as indicated in Eq. (a) of Table 5. The number of output channels is determined by bits per word in the DP memory in the proposed RLDA, as shown in Fig. 1. Eq. (a) is performance per PE, and hence total performance is proportional to the number of PEs. This means that the performance depends on the memory size and its bits per word. In the FPGA used in this work, the amount of memory (BRAM) is limited, and the unit BRAM is fixed to 72 bits per word with 512 words. In contrast, when implemented on ASIC, a larger amount of memory is available and its bits per word can be configured by designers. Thus, we understand the better performance can be achieved when the proposed RLDA-based CNN accelerator is implemented on ASIC. On the other hand, as the number

of output channels increases, the amount of transferred data from DRAM to the DP memories also increases, as indicated in Eq. (e) of Table 5. Therefore, DRAM bandwidth is also important to achieve higher performance in the proposed RLDA. In addition, the proposed accelerator does not require DSP available on the FPGA, as indicated in Table 3. This makes it easier to implement it on ASIC.

VI. CONCLUSION

In this paper, the RLDA-based CNN accelerator was proposed and implemented on FPGA. The proposed RLDA is a lookup-based computing techniques that can calculate dot products approximately without multiplication. Although lookup-based CNN accelerators have been conventionally implemented on FPGA and AISC, they have not been adopted for the ImageNet 1000 image classification because of inference accuracy degradation. In this study, we demonstrated that the proposed RLDA-based CNN accelerator can process the ImageNet task using ResNet-18 model within -5% accuracy degradation. Furthermore, the proposed accelerator was implemented on the ZCU104 evaluation board, which includes the Zynq UltraScale+ FPGA and DDR4 DRAM. We demonstrated that the processing time of the proposed accelerator on ZCU104 was comparable to that of Jetson Orin in the lower layers of the ResNet-18 model. Finally, the computational performance of the proposed accelerator was compared to that of the conventional FPGA-based accelerators. Owing to the multiplication-free architecture, DSP is not required in the proposed accelerator, which makes it easier to implement the accelerator on ASIC. We demonstrated that the performance of the proposed RLDA-based CNN accelerator was more than four times better than that of the conventional DSP-free accelerator.

ACKNOWLEDGMENT

This work was partly conducted in the AI Chip Design Open Innovation Laboratory, AIST.

REFERENCES

- [1] H. Fuketa and K. Uchiyama, "Edge artificial intelligence chips for the cyberphysical systems era," *Computer*, vol. 54, no. 1, pp. 84–88, Jan. 2021.
- [2] NVIDIA. *A100 Tensor Core GPU Datasheet*. Accessed: Jul. 25, 2024. [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf>
- [3] L. S. Karumbunathan. *NVIDIA Jetson AGX Orin Series Technical Brief*. Accessed: Jul. 25, 2024. [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/gtc21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf>
- [4] B. Wang, C. Xue, Z. Feng, Z. Zhang, H. Liu, L. Ren, X. Li, A. Yin, T. Xiong, Y. Xue, S. He, Y. Kong, Y. Zhou, A. Guo, X. Si, and J. Yang, "A 28nm horizontal-weight-shift and vertical-feature-shift-based separate-wl 6T-sram computation-in-memory unit-macro for edge depth-wise neural-networks," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2023, pp. 134–135.
- [5] S.-E. Hsieh, C.-H. Wei, C.-X. Xue, H.-W. Lin, W.-H. Tu, E.-J. Chang, K.-T. Yang, P.-H. Chen, W.-N. Liao, L. L. Low, C.-D. Lee, A.-C. Lu, J. Liang, C.-C. Cheng, and T.-H. Kang, "7.6 A 70.85–86.27TOPS/W PVT-insensitive 8B word-wise ACIM with post-processing relaxation," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2023, pp. 136–137.
- [6] G. Jedhe, C. Deshpande, S. Kumar, C.-X. Xue, Z. Guo, R. Garg, K. Soon Jway, E.-J. Chang, J. Liang, Z. Wan, and Z. Pan, "A 12nm 137 TOPS/W digital compute-in-memory using foundry 8T SRAM bitcell supporting 16 kernel weight sets for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2023, pp. 1–2.
- [7] H. Fujiwara et al., "34.4 A 3nm, 32.5TOPS/W, 55.0TOPS/mm² and 3.78Mb/mm² fully-digital compute-in-memory macro supporting INT12 × INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2024, pp. 572–573.
- [8] M. Horowitz, "1.1 Computing's energy problem (and what we can do about it)," in *Proc. IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, Feb. 2014, pp. 10–14.
- [9] R. M. Gray, "Vector quantization," *IEEE ASSP Mag.*, vols. AM-1, no. 2, pp. 4–29, Apr. 1984.
- [10] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, Jan. 2011.
- [11] J. Ran, R. Lin, J. Chun Lok Li, J. Zhou, and N. Wong, "PECAN: A product-quantized content addressable memory network," in *Proc. Design, Autom. Test Eur. Conf. Exhibition*, Apr. 2023, pp. 1–6.
- [12] X. Tang, Y. Wang, T. Cao, L. L. Zhang, Q. Chen, D. Cai, Y. Liu, and M. Yang, "LUT-NN: Empower efficient neural network inference with centroid learning and table lookup," in *Proc. 29th Annu. Int. Conf. Mobile Comput. Netw.*, Oct. 2023, pp. 1–15.
- [13] A. F. AbouElhamayed, A. Cui, J. Fernandez-Marques, N. D. Lane, and M. S. Abdelfattah, "PQA: Exploring the potential of product quantization in DNN hardware acceleration," 2023, *arXiv:2305.18334*.
- [14] J. Schönleber, L. Cavigelli, R. Andri, M. Perotti, and L. Benini, "Stella nera: Achieving 161 Top/s/W with multiplier-free DNN acceleration based on approximate matrix multiplication," 2023, *arXiv:2311.10207*.
- [15] H. Fuketa, "Lookup table-based computing-in-memory macro approximating dot products without multiplications for energy-efficient CNN inference," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 10, pp. 1–10, Oct. 2023.
- [16] B.-H. Juang and A. Gray, "Multiple stage vector quantization for speech coding," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, vol. 7, May 1982, pp. 597–600.
- [17] Y. Chen, T. Guan, and C. Wang, "Approximate nearest neighbor search by residual vector quantization," *Sensors*, vol. 10, no. 12, pp. 11259–11273, Dec. 2010.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [19] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.
- [20] A. Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 6309–6318.
- [21] Z. Hou, M. Qin, F. Sun, X. Ma, K. Yuan, Y. Xu, Y.-K. Chen, R. Jin, Y. Xie, and S.-Y. Kung, "CHEX: Channel exploration for CNN model compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 12277–12288.
- [22] ZCU104 Evaluation Board User Guide. Accessed: Jul. 25, 2024. [Online]. Available: <https://docs.amd.com/v/u/en-US/ug1267-zcu104-eval-bd>
- [23] Nsight Systems User Guide. Accessed: Jul. 25, 2024. [Online]. Available: <https://docs.nvidia.com/nsight-systems/UserGuide/index.html>
- [24] M. Huang, J. Luo, C. Ding, Z. Wei, S. Huang, and H. Yu, "An integer-only and group-vector systolic accelerator for efficiently mapping vision transformer on edge," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 5289–5301, Dec. 2023.
- [25] Y. Ma, M. Kim, Y. Cao, S. Vrudhula, and J.-S. Seo, "End-to-end scalable FPGA accelerator for deep residual networks," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2017, pp. 1–4.
- [26] A. J. A. El-Maksoud, M. Ebbad, A. H. Khalil, and H. Mostafa, "Power efficient design of high-performance convolutional neural networks hardware accelerator on FPGA: A case study with GoogLeNet," *IEEE Access*, vol. 9, pp. 151897–151911, 2021.



HIROSHI FUKETA (Member, IEEE) received the B.E. degree in electrical and electronic engineering from Kyoto University, Kyoto, Japan, in 2002, and the M.E. and Ph.D. degrees in information systems engineering from Osaka University, Osaka, Japan, in 2008 and 2010, respectively.

From 2010 to 2015, he was a Research Associate with the Institute of Industrial Science, The University of Tokyo, Tokyo, Japan. Since April 2015, he has been with the National Institute of Advanced Industrial Science and Technology (AIST), Ibaraki, Japan. His research interests include the circuit design of AI chips and cryogenic CMOS circuits for quantum computers. He has served as a TPC Member for the IEEE International Solid-State Circuits Conference (ISSCC), from 2016 to 2018. He is a TPC Member of the IEEE Asian Solid-State Circuits Conference (A-SSCC).



YOHEI HORI (Member, IEEE) received the B.E., M.E., and Ph.D. degrees from the University of Tsukuba, Ibaraki, Japan, in 1999, 2001, and 2004, respectively. After receiving the Ph.D. degree, he spent five years as a Postdoctoral Researcher with the National Institute of Advanced Industrial Science and Technology (AIST). He moved to Chuo University as a Research Scientist, in 2008, before returning to AIST, in 2010, as a Researcher. He is currently a Senior Researcher with the Semi-

conductor Frontier Research Institute (SFRC), AIST. His current research interests include the design and architecture of reconfigurable systems, open-source EDA tools, and hardware security. He is a member of IEICE and IPSJ.



TOSHIHIRO KATASHITA received the Ph.D. degree from the University of Tsukuba, in 2006. In 2006, he joined the National Institute of Advanced Industrial Science and Technology (AIST) as a fixed-term Researcher. In 2008, he joined AIST as a tenure-track Researcher. He is currently involved in research projects on high-performance computation circuit design and on hardware security. He is a member of IEICE and IPSJ.



MASAKAZU HIOKI (Member, IEEE) received the B.E., M.E., and Ph.D. degrees in electronics engineering from Tohoku University, Japan, in 1998, 2000, and 2003, respectively. He joined the Electoinformatics Group, Nanoelectronics Research Institute, National Institute of Advanced Industrial Science and Technology (AIST), Japan, in 2003, working on an architecture design and a circuit design of power reconfigurable FPGA with fine-grained body biasing, called Flex Power

FPGA. Since 2019, he has been the Team Leader of the AI chip Design Open Innovation Laboratory (AIDL), AIST. Since 2020, he has been the Group Leader of the Advanced Integrated Circuit Research Group, Device Technology Research Institute (D-Tech), AIST. He is currently the Team Leader of the Integrated Circuit Design Research Team, Semiconductor Frontier Research Center (SFRC), AIST. His research interests include the architecture and circuit design of reconfigurable systems, low-power circuit design, and non-volatile memory.

...